



# SIMATS Online Programming Lab

## JAVA PROGRAMMING



NITHISH RAAJU V  
192411016RD

### QUESTIONS

Q1

Multiple Threads Using a Single Class

10 marks



Q2

Multiple thread classes

20 marks



Q3

Thread Synchronization

10 marks



3/3 answered

Q1

### Multiple Threads Using a Single Class

10  
marks

Topic: Thread class, multiple threads, if-else

Question:

Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.

Armstrong → Check whether the number is an Armstrong number.

Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number

Armstrong Thread: 153 is an Armstrong number

Reverse Thread: Reverse of 153 = 351

## Your Code

```
1 public class ThreadExample {
2     public static void main(String[] args)
3         MyThread t1 = new MyThread("Prime")
4         MyThread t2 = new MyThread("Armstrong")
5         MyThread t3 = new MyThread("Reverse")
6
7         t1.start();
8         t1.join();
9
10        t2.start();
11        t2.join();
12
13        t3.start();
14        t3.join();
15    }
16 }
17
18 class MyThread extends Thread {
19     int number = 153;
20
21     MyThread(String name) {
22         super(name);
23     }
24
25     public void run() {
26         String name = this.getName();
27
28         if (name.equals("Prime")) {
29             checkPrime(number);
30         } else if (name.equals("Armstrong")) {
31             checkArmstrong(number);
32         } else if (name.equals("Reverse")) {
33             reverseNumber(number);
34         }
35     }
36
37     void checkPrime(int n) {
38         boolean prime = true;
39         if (n < 2) {
40             prime = false;
41         } else {
42             for (int i = 2; i <= Math.sqrt(n); i++) {
43                 if (n % i == 0) {
44                     prime = false;
45                     break;
46                 }
47             }
48         }
49     }
50 }
```

```
47         }
48     }
49     if (prime) {
50         System.out.println("Prime Thread");
51     } else {
52         System.out.println("Prime Thread");
53     }
54 }
55
56 void checkArmstrong(int n) {
57     int original = n;
58     int sum = 0;
59     int digits = String.valueOf(n).length();
60     while (n > 0) {
61         int digit = n % 10;
62         sum += Math.pow(digit, digits);
63         n /= 10;
64     }
65     if (sum == original) {
66         System.out.println("Armstrong");
67     } else {
68         System.out.println("Not Armstrong");
69     }
70 }
71
72 void reverseNumber(int n) {
73     int original = n;
74     int rev = 0;
75     while (n > 0) {
76         int digit = n % 10;
77         rev = rev * 10 + digit;
78         n /= 10;
79     }
80     System.out.println("Reverse Thread");
81 }
82 }
```

### Input

153

## Output

```
Prime Thread: 153 is not a prime number  
Armstrong Thread: 153 is an Armstrong number  
Reverse Thread: Reverse of 153 = 351
```

## Visible Test Cases

|             |   |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

**10.00 / 10 marks**

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.